

Nataraja

Cloud-Native LLM Orchestration Platform — a depth-first learning & build roadmap taking one engineer from senior DevOps/SRE to Cloud Solution Architect, built one running increment at a time.

56 weeks (~13 months) end-to-end	14 build stages across 12 phases
8–10 study + build hours per week	18 technology domains covered
30+ Architecture Decision Records (ADRs)	3 "go-to-90%" mastery anchors

01 What Nataraja is & how it's built

Nataraja is an asynchronous **job-orchestration platform / LLM gateway**: clients submit jobs, a gateway accepts them instantly (202 + id), routes to a healthy provider under rate limits, workers process with retry/fallback, every state change is event-sourced, with multi-tenant quotas and full observability — deployed on Kubernetes and AWS. It begins as a 5-endpoint Flask CRUD and is scaled, stage by stage, into a distributed, event-driven, multi-cloud system. The purpose is to **learn and be able to defend** distributed systems, resilience, data-at-scale, and cloud architecture — not merely to ship an app.

The learning method — every stage runs the same four-beat loop

BEAT 1

Read before building

Hit the concepts & curated readings first. You must explain *why* the thing exists before touching the keyboard.

BEAT 2

Design in prose

Model, API, and state machine designed in plain English and pressure-tested — before any implementation.

BEAT 3

Build it yourself

100% of the code is written by hand. The point is genuine understanding, not generated output.

BEAT 4

Review + ADR

Hunt race conditions and weak tradeoffs, defend every "why," then record a one-page ADR. Target: 30+.

Pacing philosophy — spine first. The plan is calibrated to a senior DevOps background: it does **not** teach all domains from zero. Roughly **60% of the hours land in Weeks 1–29** (Stages 0.5→5) — application Python, distributed-systems implementation, the agent layer, and data-at-scale, the genuine gaps on the road to architect. The back half (Kubernetes deep, observability, CI/CD, Terraform) moves fast because it builds on skills already operated daily. **Golden rule: one stage at a time; it must run before the next begins.**

DEEP-NEW genuine gaps — full time **EXTEND** adjacent — moderate **LEVERAGE** already operated — showcase

MIXED part new, part known ★ mastery anchor

02 The 56-week roadmap

WEEKS	STAGE	WHAT YOU BUILD & THE CONCEPTS BEHIND IT	TIER
PHASE A · Language & app discipline — Wks 1–7			
1–4	0.5	Python + testing foundations Typed async service module + real DB migration. Own: typing/Pydantic v2, async/await & the event loop, pytest, hypothesis, async SQLAlchemy + Alembic, uv packaging.	DEEP-NEW
5–7	0	FastAPI rewrite + clean architecture Hexagonal (ports/adapters), validation, idempotency keys, optimistic concurrency, correlation IDs, 12-factor config.	DEEP-NEW
PHASE B · The distributed core — Wks 8–14			
8–11	1	Async core: queue, state machine, event sourcing 202+id submit, durable queue (Redis Streams), worker, job state machine, append-only event log, idempotent consumer, cancellation.	DEEP-NEW
12–14	2	Polyglot persistence: MongoDB + Redis Postgres relational core; trajectories → MongoDB; Redis hot state. Embed-vs-reference modeling, aggregation, change streams, TTL.	EXTEND
PHASE C · The agent layer — Wks 15–19			
15–19	3	Agent loop + RAG ★ — Emergent core Real agent loop with Claude (plan→tool-call→observe), token budgets, trajectory persistence, SSE streaming, multi-agent patterns, RAG (embeddings + vector DB), eval harness.	DEEP-NEW
PHASE D · Resilience — Wks 20–24			
20–24	4	Rate-limit / breaker / retry / fallback ★ — Anchor #1 Provider registry & selector, distributed rate limiter (token bucket + sliding window via Redis/Lua), circuit breaker, retry+backoff+jitter, fallback chain, load shedding.	DEEP-NEW
PHASE E · Event-driven backbone — Wks 25–29			
25–29	5	Kafka + CQRS + Saga + CDC Kafka/Redpanda backbone — partitions, consumer groups, exactly-once, DLQ; schema registry; outbox; CQRS read models; Saga; CDC via Debezium. End of the spine — Week 29.	DEEP-NEW
PHASE F · Multi-tenancy & identity — Wks 30–32			
30–32	6	Tenants, identity, Vault Tenants, API keys, quotas, usage ledger; OAuth2/OIDC/JWT (Keycloak); Vault dynamic creds + PKI; RBAC; prompt-injection defense.	EXTEND
PHASE G · Internal comms & Go — Wks 33–36			
33–36	7	gRPC, service mesh, Go gRPC + Protobuf between services; worker rewritten in Go (goroutines/channels/context); Istio for mTLS + traffic-shaping.	MIXED
PHASE H · Kubernetes deep + custom Operator — Wks 37–40			

WEEKS	STAGE	WHAT YOU BUILD & THE CONCEPTS BEHIND IT	TIER
37–40	8	K8s deep + custom Operator + KEDA ★ — Anchor #2 RBAC/NetPol/PSS/PDB, plus a custom Operator + CRD (Kubebuilder), KEDA autoscaling on Kafka lag, Kyverno/OPA policy-as-code.	LEVERAGE
PHASE I · Frontend + experimentation — Wks 41–44			
41–44	9	Dashboard + A/B framework Next.js/React/TS dashboard, live trajectory (SSE); experimentation framework — feature flags, traffic splitting, statistical significance, guardrails/auto-rollback.	MIXED
PHASE J · Preview environments — Wks 45–47			
45–47	10	Per-tenant preview environments Ephemeral isolated stacks (vcluster), external-dns + cert-manager for domains/TLS, ArgoCD ApplicationSets — powered by the Stage-8 operator.	EXTEND
PHASE K · Cloud, IaC, multi-region — Wks 48–52			
48–52	11	Terraform AWS + multi-region Terraform a real AWS env (VPC/EKS+IRSA/RDS/ElastiCache/MSK/S3/ALB/Route53/CloudFront/ACM/WAF/KMS); multi-region active-passive→active-active; FinOps (Infracost).	MIXED
PHASE L · Observability, SRE, chaos & capstone — Wks 53–56			
53–56	12	Instrument, SLOs, chaos, supply-chain, capstone ★ — Anchor #3 OTel across async/Kafka/gRPC; Prometheus + Loki + Grafana + Pyroscope; SLOs + burn-rate alerts; chaos (Chaos Mesh); supply-chain (SBOM/cosign/SLSA); k6 load test; capstone artifacts.	LEVERAGE

03 Technology & tools catalog

The complete set of technologies, tools, and concepts covered across the journey, grouped into 18 domains. Each domain notes the stage(s) where it is built. **Highlighted** items are the primary, hands-on tools of that domain.

1 · Languages & service communication

Stages 0, 1, 3, 7, 9

Python 3.12+ **FastAPI** **async/await** **Go** **gRPC** **Protobuf** REST GraphQL (BFF) OpenAPI contract testing
Rust / Java (optional)

2 · Python & testing foundations

Stage 0.5

type hints / typing **Pydantic v2** **pytest** **hypothesis** **SQLAlchemy (async)** **Alembic** **uv** mypy context managers
decorators packaging (pyproject)

3 · Agent / LLM layer

Stage 3

Anthropic Claude API **agentic loops (plan-act-observe)** **tool / function calling** **RAG** **embeddings + vector search**
token budgeting context-window mgmt multi-agent patterns SSE streaming eval harness prompt-injection defense
LLM cost control

4 · Data & storage (polyglot persistence)

Stages 1, 2, 3, 5

PostgreSQL **MongoDB** **Redis** **pgvector / Qdrant** S3 / MinIO (object) ClickHouse / DuckDB / BigQuery (OLAP)
Debezium (CDC) PgBouncer indexing / partitioning / replication

5 · Messaging & event-driven architecture

Stages 1, 5

Kafka / Redpanda **Schema Registry** **event sourcing** **CQRS** **Saga** outbox pattern DLQ partitions / consumer groups
exactly-once semantics stream processing

6 · Distributed systems

Stages 1, 4, 5, 11

consensus (Raft / etcd) **distributed rate limiting** leader election distributed locking consistent hashing quorums & replication
vector clocks / ordering CAP / PACELC

7 · Resilience & reliability

Stages 4, 12

circuit breaker **retry / backoff / jitter** **rate limiting** **load shedding** bulkhead backpressure graceful degradation
chaos engineering (Chaos Mesh / Litmus) fault injection

8 · Containers & orchestration

Stages 8, 10

Docker **Kubernetes** **custom Operator + CRDs** **KEDA** **Helm** **Kustomize** RBAC / NetworkPolicies / PodSecurity
StatefulSets HPA / VPA Kyverno / OPA Gatekeeper multi-cluster

9 · CI/CD, GitOps & progressive delivery

Stages 8, 9, 12

GitHub Actions **Argo CD** **Argo Rollouts** **OIDC (keyless deploy)** Argo Workflows / Tekton canary / blue-green feature flags
SBOM cosign / sigstore SLSA provenance policy-as-code

10 · Infrastructure as Code

Stages 8, 10, 11

Terraform **Terratest** **Ansible** Terragrunt remote state Crossplane Pulumi (optional)

11 · Cloud (AWS-first, multi-cloud)

Stage 11

AWS EKS **VPC** **IAM / IRSA** **RDS** ElastiCache MSK / SQS S3 Secrets Manager ALB / NLB Route 53
CloudFront ACM WAF KMS multi-region (active-passive → active-active) GKE Cloudflare R2 / Workers FinOps / Infracost
Well-Architected

12 · Networking

Stages 7, 10, 11

VPC / subnets / NAT / IGW **ingress controllers** **cert-manager / ACM / mTLS** SG vs NACL PrivateLink / VPC endpoints
Transit Gateway / peering DNS (Route 53) L4 / L7 load balancing API gateway zero-trust

13 · Security & identity

Stages 6, 11

OAuth2 / OIDC / JWT **Keycloak** **HashiCorp Vault** IAM least-privilege RBAC dynamic secrets + PKI External Secrets
KMS / envelope encryption OWASP API Top 10 secret scanning SAST / DAST

14 · Service mesh

Stage 7

Istio Linkerd mTLS traffic shaping mesh retries / timeouts mesh-level canary

15 · Observability & SRE

Stage 12

Prometheus **OpenTelemetry** **Grafana** **Loki** **SLO / SLI / error budgets** Alertmanager Tempo / Jaeger (tracing)
Pyroscope (profiling) eBPF (Cilium / Pixie) burn-rate alerts runbooks / postmortems PagerDuty

16 · Scalability, performance & data engineering

Stages 9, 12

k6

A/B experimentation framework

Locust / Gatling

capacity planning

multi-layer caching

read replicas / sharding

tail-latency optimization

analytics pipeline

17 · Frontend

Stage 9

Next.js

React

TypeScript

SSE / streaming UI

18 · Architecture & communication

All stages · capstone

C4 diagrams

ADRs (30+)

Well-Architected reviews

design docs / RFCs

cost models

DR runbooks

failure playbooks

scale-at-100x analysis

04 Certifications & capstone deliverables

Certifications are woven into the timeline, not bolted on. The capstone artifacts are what turn "an engineer who deployed an app" into "an architect with a system to point at."

Certifications (targeted alongside the build)

PHASE B · WKS ~8–14 CKA (+ optional CKAD) Kubernetes Administrator — a fast confidence win given daily K8s operation; CKAD follows now that apps are being written for the cluster.	PHASE K · WKS ~48–52 AWS SAP-C02 Solutions Architect <i>Professional</i> — matched to the depth built in the cloud/multi-region stage. SAA-C03 only as a stepping stone if breadth is rusty.
--	---

Capstone portfolio — the definition of done

- A running **multi-region, multi-cloud, agentic platform** spanning 3 languages and 5+ datastore types.
- **30+ ADRs** — a decision record for every real architectural choice.
- A **cost model** and FinOps analysis of the running system.
- **DR runbooks** and a **failure playbook**.
- All three flagship designs: **agent orchestration, experimentation, preview environments**.
- A **Well-Architected review** across all six pillars.
- **C4 + "Exceptional-4" diagrams** of the full architecture.
- A **scale-at-100x** analysis and stakeholder-facing design doc.

Nataraja — Cloud-Native LLM Orchestration Platform · Learning & Build Roadmap · calibrated to a senior DevOps/SRE background, ~8–10 hrs/week over ~56 weeks · one running increment per stage.